

回転機構のレーザー受光と楽器間無線 通信による同期・輪唱システム

情報工学科 2 年

24G1059

佐藤 涼菜

—— チーム 40 ——

宇井 祐真 (24G1021) 梅野 大誠 (24G1026)

山口 侑希 (24G1136) 薄井 悠希 (25G1020)

菊池 侑人 (25G1041)

2026 年 7 月 9 日

1 はじめに

1.1 社会的背景・課題・本稿の目的

近年、動画配信サイトやストリーミングサービスの普及によって、誰もが手軽に音楽コンテンツを視聴できる環境が整っている。特に日本では、ゲームやアニメなどのサブカルチャーを起点とした音楽コンテンツが数多く生み出されており、独自の文化として定着している。一方で、このようなデジタル配信の消費形態は、実際に会場へ足を運んで演奏を体感するライブイベントやコンサートとの相乗効果を生み、市場規模の拡大にも寄与している [1][2]。たとえば、博報堂の谷口による分析では、ストリーミングサービスの利用者がサービス内で新たに好みのアーティストを複数発掘し、そのアーティストたちを一度に鑑賞できるライブやフェスへ参加するという行動の流れが形成されつつあり、さらにライブでの体験を通じて別のアーティストと出会うことで、再びストリーミングでの視聴に戻るという音楽消費サイクルが生まれていると指摘されている [4]。すなわち、デジタル配信とライブイベントは互いの顧客を送客し合う関係にあり、この循環が音楽市場全体の拡大を後押ししていると考えられる。実際に、一般社団法人コンサートプロモーターズ協会が実施したライブ市場調査によれば、2025 年におけるライブ市場の年間売上高は 6443 億円、入場者数は 5999 万人にのぼり、デジタル配信の普及以降もライブ市場は堅調に拡大を続けている [3]。図 1 に示すように、国内のライブ・コンサート市場における年間売上額は 2010 年の 1280 億円から 2019 年の 3665 億円まで、10 年間でおよそ 2.9 倍に拡大している。2020 年には新型コロナウイルス感染症の影響により、会場に人を集めること自体が困難となった結果、年間売上額は 779 億円まで急落した。しかし、行動制限の緩和とともに市場は急速に回復し、2022 年には 3984 億円、2023 年には 5140 億円、そして 2025 年には過去最高となる 6443 億円を記録している。この推移から、ライブ会場に足を運ぶという体験に対する需要は、一時的な外的要因によって落ち込むことはあっても、長期的には一貫して拡大する傾向にあると言える。

デジタル配信が普及していく中で、なぜデジタル配信では代替できない価値がライブ会場に存在するのだろうか。その理由の一つとして、ライブ演出に用いられる音響や照明、ライブ参加者自身が感じる振動といった複数の物理的刺激が挙げられる。これらの刺激は、観客一人ひとりに個別に届けられるのではなく、同じ空間にいる観客全員に同時に共有されるという性質を持つ。その結果、観客同士が同じ体験を共有しているという感覚、すなわち空間全体としての一体感が形成される。換言すれば、ライブ会場における没入感とは、ユーザが音楽を聴くだけでとどまらない、物理的、視覚的な技術と同じ空間で同時に受け取ることによって生まれる体験だと考えられる。この仮説は、複数の研究によっても裏付けられている。ライブ演奏とデジタル配信を直接比較した実証研究によっても裏付けられている。Kreuzer ら (2025) は、同一のクラシック音楽コンサートを、会場で聴取する条件と映像配信で視聴する条件とで比較する実験を行った。その結果、没入感や社会的つながりの感覚を含む複数の評価軸において、ライブ条件の方が配信条件よりも有意に高い評価を得ることを示した [5]。すなわち、同じ演奏内容・同じ音質であっても、同じ空間を他の観客

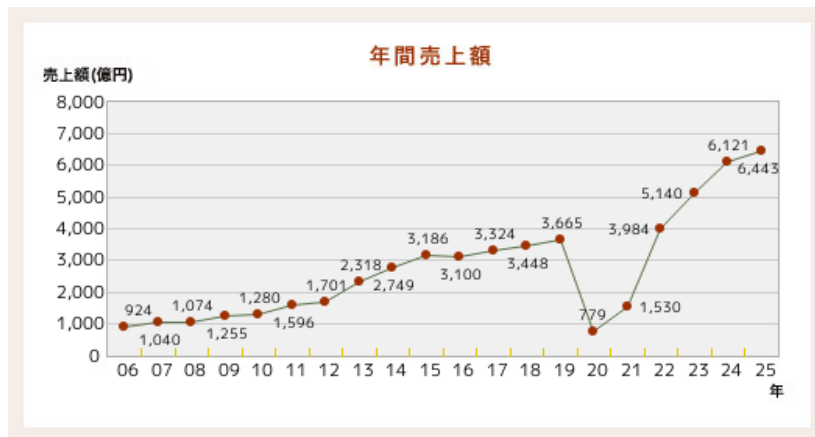


図1 国内ライブ・コンサート市場における年間売上額の推移（一般社団法人コンサートプロモーターズ協会「基礎調査推移表」より）

と共有しているという感覚そのものが、没入感を左右する独立した要因であり、これは録画・配信技術の高度化だけでは補えないことを意味する。したがって、音そのものの再現性だけでなく、ライブ体験に固有の空間的フィードバックを組み込むことで没入感の高い演奏体験の実現に有効であると考えられる。

また、Natalia Cooper ら（2018）は、仮想現実（VR）環境における体験の再現度を高めるため、視覚情報のみでは表現が難しい感覚を、音や触覚など別の感覚を用いた代替フィードバックとして提示する実験を行った。その結果、視覚刺激のみを提示した場合と比較して、音や触覚といった複数の異なる刺激を組み合わせると提示した場合の方が、利用者の没入感・関与感が有意に高まることを明らかにした [6]。つまり、複数の感覚情報を同時に提示する多感覚的フィードバックは、ユーザの体験における没入感の向上に有効であると考えられる。また、Human-Computer Interaction (HCI) 分野の知見も同様の傾向を支持している。Martin ら（2024）は、ライブコンサート環境を対象とした研究において、観客が演奏を受動的に聴取するだけでなく、能動的に体験へ参加することが、会場全体の活気や観客の感情の高まりに寄与すると結論づけた [7]。よって、ユーザ自身が能動的に関与できる仕組みもまた、システムの没入感を高める要因として重要であると考えられる。

一方、音楽を自動で演奏する自動演奏技術の分野では、デジタル技術の発展により高精度な同期や楽曲再現が可能になっている。たとえば、Yamaha が提供する Disklavier ENSPIRE シリーズは、光学センサとサーボモータを用いて演奏情報を高精度に再現するとともに、鍵盤が実際に動作する様子をユーザへ視覚的にフィードバックを行う機能を備えている [8]。しかし、この視覚的フィードバックは、ピアノの鍵盤という限られた範囲でのみ提示されるものであり、その場に居合わせたユーザ1人のみが確認できる情報にとどまる。したがって、複数人が同じ空間にいる状況を想定すると、自動演奏技術によって提示される視覚的フィードバックは、空間全体には行き渡らないという限界を抱えていると言える。

以上を踏まえると、現状の自動演奏技術は、演奏そのものの精度は高い一方で、前述した「多感覚的フィードバック」と「空間全体への共有」という、没入感を高めるうえで重要な要素を十分に

は満たせていないという課題が生じる。特に、Kreuzer らの知見が示すとおり、同じ場を共有しているという空間的なフィードバックは、音質や演奏精度をいくら高めても代替できない、ライブ体験に固有の要素である。したがって、デジタル配信や高精度な自動演奏技術がどれほど発展しても、この空間的フィードバックを欠いたままでは、ライブ会場に匹敵する高い没入感を得ることはできないと結論づけられる。そこで本稿では、自動演奏技術に、ライブ体験に共通するこの空間的フィードバックを付加した楽器間通信システム「回転機構のレーザー受光と楽器間無線通信による同期・輪唱システム」を提案する。本システムは、ユーザの動作を検知する指揮デバイス、指揮デバイスからの情報に応じ回転する観覧車型の中継機デバイス、そして中継機からのレーザー光を受光し演奏を行う楽器デバイスによって構成される。指揮デバイスによるユーザの能動的な動作、観覧車型デバイスによる空間全体への視覚的なフィードバックの共有、そして高精度な音色の演奏による聴覚的な刺激を組み合わせることで、従来の自動演奏技術では実現が難しかった、会場全体を巻き込む没入感の高い演奏体験の提供を目指す。

1.2 類似技術・先行研究

本節では、本システムの独自性を明確にするため、音楽に視覚的なフィードバックを付加する既存技術を整理し、それぞれの特徴と限界を検討する。

第一に、DMX512 信号（以下、DMX 信号と表記する）を用いた照明制御システムが挙げられる。DMX 信号とは、舞台照明の操作卓と調光機との間でデータをやり取りするための通信規格であり、現在では舞台演出やコンサートにおいて、音楽に合わせた光量制御を行う目的で広く利用されている [9]。このシステムは、音楽という聴覚的な刺激に加えて光という視覚的な刺激を提示することで、観客の没入感を高めることができる。しかし、DMX 信号による照明制御は、あらかじめプログラムされた演出を再生する仕組みであるため、観客はその光の変化を一方的に受け取る、すなわち受動的に認識する立場にとどまる。前節で述べたとおり、能動的な関与が没入感を高める要因の一つであることを踏まえると、観客が演奏そのものに関与できない DMX 信号による演出には限界があると言える。この課題に対し、本提案では指揮デバイスを導入し、利用者が指揮という身体動作を通じて演奏そのものを能動的に制御できるようにすることで、身体的な関与に基づく没入感の向上を狙う。

第二に、物理的な操作によって音楽を制御するタンジブル・インターフェースの例として、ReacTable が挙げられる。ReacTable は、卓型のディスプレイ上に複数の物理モジュールを配置し、その配置や動きに応じて音を合成するシンセサイザの一種であり、利用者は身体的な物理操作を通じて演奏を制御するとともに、ディスプレイに表示される視覚的な情報によって演奏状態のフィードバックを受け取ることができる [10]。しかし、この視覚的フィードバックはディスプレイという限られた表示領域内で完結しており、操作を行う利用者本人以外の第三者や、会場全体へフィードバックを共有する仕組みとしては設計されていない。そこで本提案では、指揮デバイスに加えて観覧車型の中継機デバイスを導入する。観覧車型デバイスが演奏情報に応じて回転する様子を、利用者本人だけでなく周囲にいる第三者も視認できるようにすることで、演奏状態の把握や共有を空間全体に広げ、多感覚的なフィードバックを実現することを狙う。

以上のように、DMX 信号による照明制御は観客の能動的な関与を欠き、ReacTable のような既存のタンジブル・インターフェースは視覚的フィードバックが個人の操作範囲内にとどまるという、それぞれ異なる限界を抱えている。本提案は、指揮デバイスによる能動的な演奏制御という既存研究の知見と、観覧車型デバイスによる空間共有性という要素を組み合わせることで、これら二つの課題を同時に解決し、空間全体への多感覚的なフィードバックを通じて、利用者の没入感および関与感を高めることを目指す。

2 作成したシステムの概要

本節では、作成したシステムの概要について説明する。本システムは、ユーザの能動的な動作と連動した観覧車型中継機デバイスの回転およびレーザーの発光を通じて、演奏空間全体に多感覚なフィードバックを与えることで、従来の自動演奏技術における演奏インターフェースでは得られない高い没入感をユーザに提供することを目的として設計されている。本システムは、指揮デバイス、観覧車型の中継機デバイス、楽器デバイスという3つのデバイスから構成される。演奏開始後、ユーザがスライド式可変抵抗器を操作したうえで指揮デバイスを振ると、加速度センサが振り動作を検知し、中継機デバイスが

ここからは各デバイスの概要について説明する。

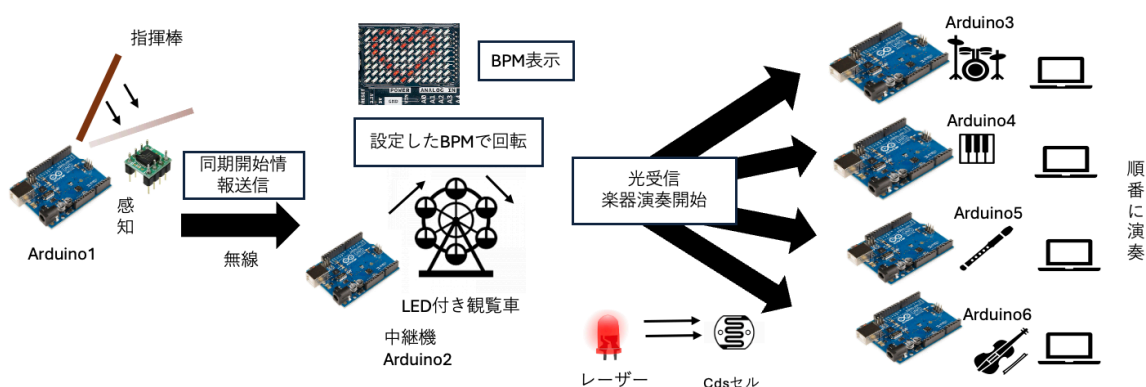


図2 全体の構成図

2.1 指揮デバイス

2.1.1 概要

まず、指揮デバイスについて説明する。指揮デバイスのシステム構成図を図3に示す。本デバイスは、演奏全体を統括するデバイスであり、主に以下の3つの機能を有する。

第一の機能は、演奏開始トリガーの送信である。本機能の設計について図4に示す。ユーザが指揮デバイスに搭載されたタクトスイッチを押下すると、中継機デバイスと楽器デバイスへ演奏開始トリガーを送信する。さらに、中継機デバイスの回転が開始される。

第二の機能は、楽曲のBPM変更である。本機能の設計について図5に示す。ユーザがスライド式可変抵抗器を操作した後、デバイスを振ると新しい設定BPM情報が中継機デバイスへ送信される。これにより、中継機デバイスのモータ回転速度がこの値に追従して変化する。そして、その回転に伴うレーザーの通過光を介して各楽器デバイスに情報が伝達され、BPMの変更される。また、楽曲のBPMは5段階に分けて変更できる。

第三の機能は、楽曲の音量変更である。本機能の設計について図6に示す。ユーザがスライド式可変抵抗器を操作した後、デバイスを振ると音量情報が各楽器デバイスへマルチキャスト送信されることで、音量が変更される。また、BPMと同様に音量も5段階に分けて変更できる。ここからは、外部設計と内部設計について説明をする。

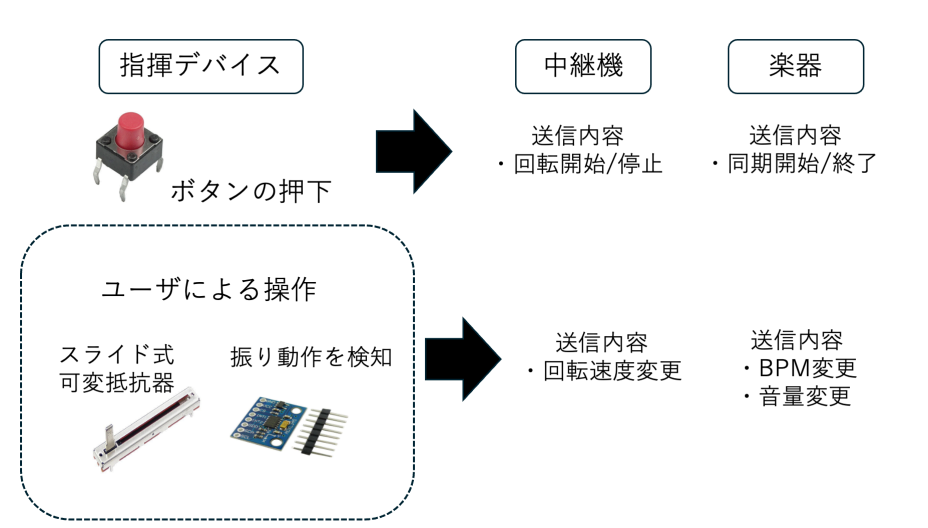


図3 指揮デバイスのシステム構成図

2.1.2 外部設計

本節では指揮デバイスの外部設計について説明する。まず、本デバイスを構成する部品について表1に示す。デジタル3軸加速度センサは指揮デバイスの動きの検知に使用し、スライド式可変抵抗器はBPMおよび音量変更に使用される。当初はデバイスの電力供給源として9Vのアルカリ電池を外部電源として使用することを検討していたが、電池なしでも正常動作が確認できたため、軽量化のためPCとのUSB接続から電力を供給するように変更した。Arduino内蔵の電圧レギュレータにより5Vおよび3.3Vを生成し、各モジュールへ安定した電力を供給する。また、指揮デバイスの回路図を図7に示す。加速度センサにはGY-291(ADXL345)を用いており、I2C通信によりArduinoと接続する。CSピンをArduinoの3.3Vに接続することでI2Cモードに設定し、さらにSDOピンをGNDに接続することでI2Cアドレスを0x53に固定している。また、SDAピンおよび

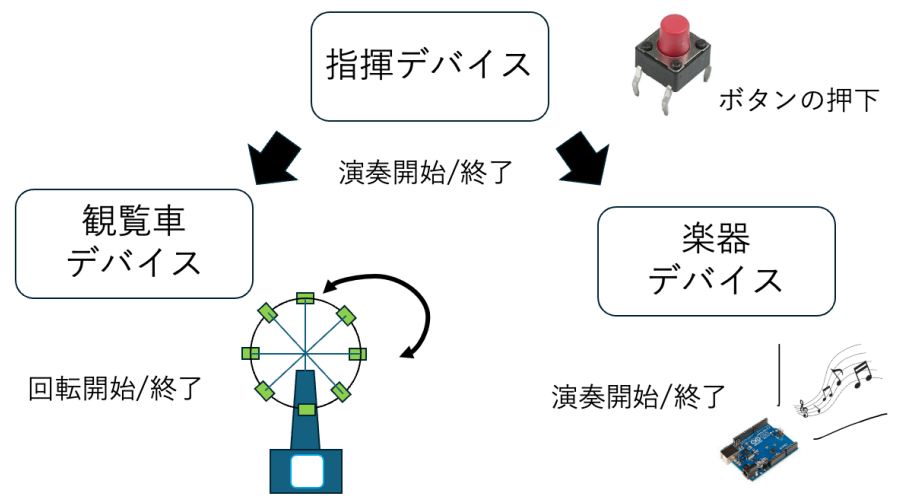


図 4 演奏開始トリガー送信機能設計

SCL ピンはそれぞれデータ通信線およびクロック信号線として Arduino の A4, A5 ピンに接続している。加えて、加速度が内部設定した閾値を超過した場合には、INT1 ピンから Arduino の D2 ピンへ割り込み信号が出力される。これにより、指揮デバイスの振り動作を検知し、モータ制御用 Arduino との UDP 通信を開始する。

さらに、演奏開始および終了信号の送信に用いるボタン入力判定回路を実装する。ボタンの誤判定を防止するため 10k Ω のプルアップ抵抗を用いた回路を構成する。ボタンが押されていない状態では、入力ピン D3 はプルアップ抵抗を介して 5V に接続されるため HIGH 状態となる。一方、ボタン押下時には回路が GND に短絡され、入力電位が LOW に遷移する。指揮デバイスのプログラムでは D3 ピンの状態変化を常時監視することで、演奏開始および終了操作を判定する。

表 1 指揮デバイスの構成部品

Arduino UNO R4 WiFi	-	1
スライド式可変抵抗器 10 k Ω	-	2
デジタル 3 軸加速度センサ	GY-291	1
タクトスイッチ	DTS-63	1
抵抗器 10k Ω	-	1
ジャンパーワイヤー	-	適宜

図～～～に実際に作成した指揮デバイスを示す。実際の指揮棒のようにユーザが片手で振って操作できるよう、細長い筒状の外装を採用した。また、内部にはブレッドボードに加速度センサ、スライド式可変抵抗器、タクトスイッチを設置する。デバイスのユーザビリティを向上させるため、ユーザが操作するスライダー部分やボタンは外部へ露出させる構造を採用し、さらにデバイスの先端部に加速度センサを設置することで操作時の重心を安定させる。

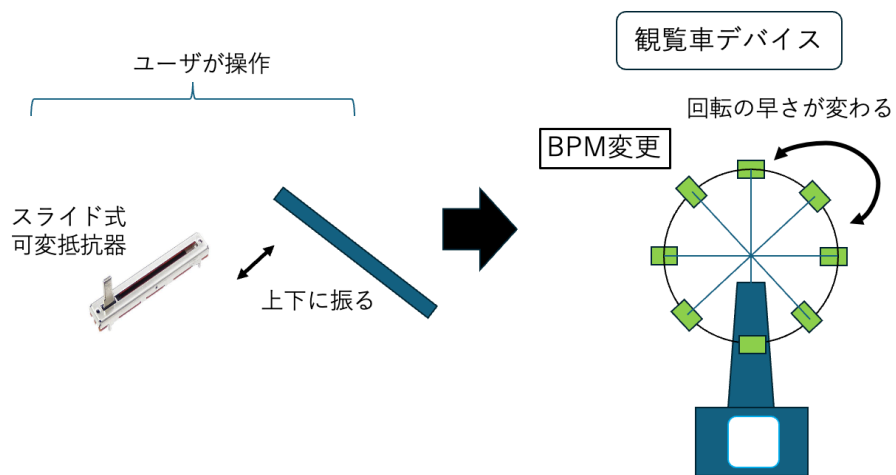


図 5 BPM 変更機能設計

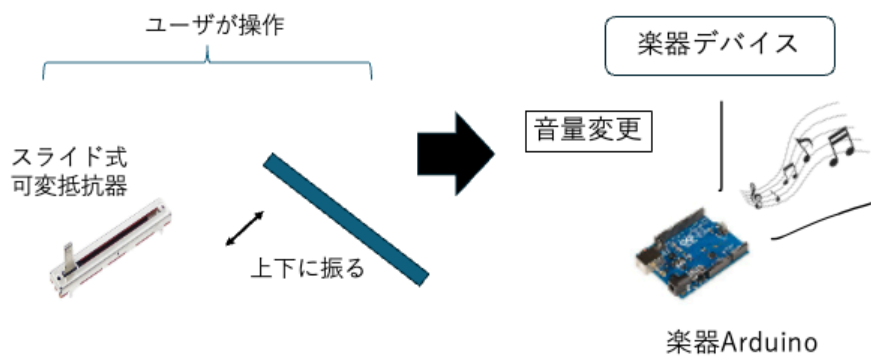


図 6 音量変更機能設計

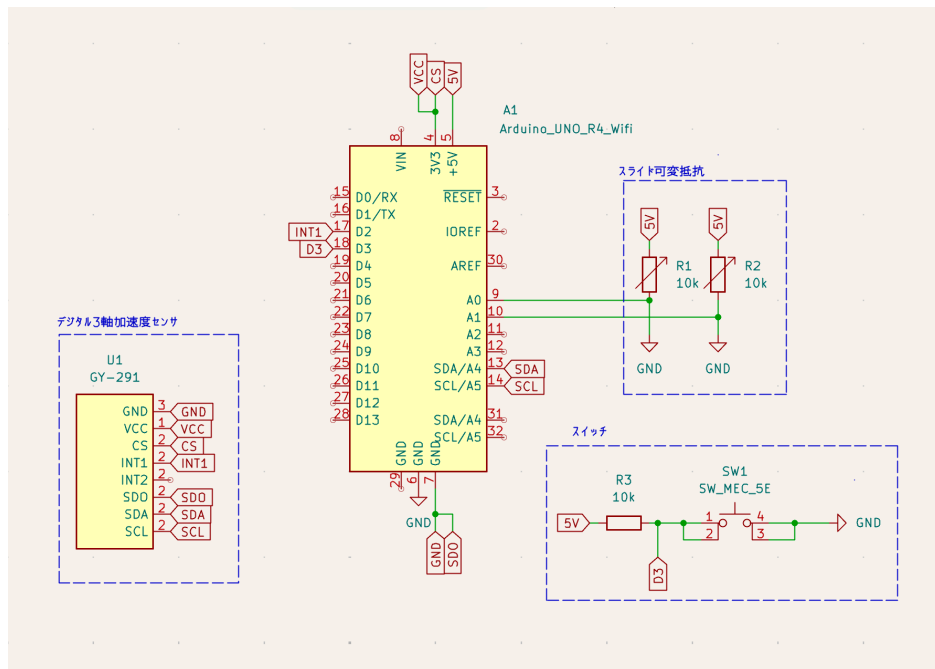


図7 指揮デバイスの回路図

2.1.3 内部設計

本節では、指揮デバイスの内部設計について説明する。まず、指揮デバイスにおけるシーケンス図を図8に示す。ユーザが指揮デバイスを上下に振ると加速度センサが動作を検知し Arduino に送信される。この時、値が閾値を超過した場合、観覧車デバイスと無線通信を開始し観覧車デバイスが送信した演奏情報を基に回転する。また、ボタンを押下した時、各楽器デバイスへ楽曲の初期BPM および音量情報が送信されることで演奏が開始される。

2.2 中継機デバイス

2.2.1 概要

次に、中継機デバイスについて説明する。中継機デバイスの構成図を図9に示す。本デバイスは、指揮デバイスからの情報を受け取り中継機デバイスの回転を作動させ、現在のBPMを表示するデバイスであり、主に以下の2つの機能を有する。

第一の機能は、上記で述べたBPMに合わせて回転し楽曲のBPMを制御する機能である。デバイスに設置してあるレーザー受光のテンポでBPMを管理し、楽曲のテンポを視覚的にも理解できるようにする。

第二の機能は、Arduino Uno R4 WiFi に付属している LEDMatrix に BPM を表示する機能である。ここに指揮デバイスから受け取った BPM を表示し、視覚的に現在流れている楽曲の BPM を理解できるようにする。ここからは、外部設計と内部設計について説明する。

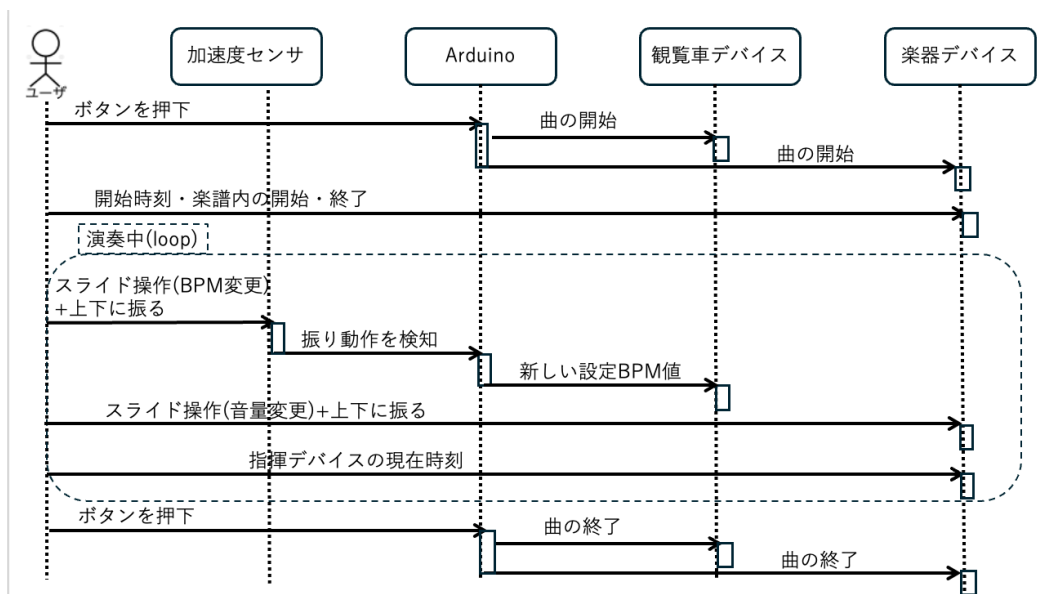


図 8 指揮デバイスのシーケンス図

2.2.2 外部設計

本節では中継機デバイスの外部設計について説明する。まず、本デバイスを構成する部分について表 2 に示す。この内、個別で購入する必要があるものはハイパワーギヤーボックス、ブレッドボード用 DC ジャック DIP 化キット、ボタン電池、ボタン電池ホルダー、QI ケーブル、有極コンデンサーである。

また、中継機の回路図を図 10 に示す。モータードライバーに Arduino D5, D6 ピンを接続し、PWM 制御を行えるようにする。モータードライバーへの電力供給は当初、外部電源として 9V のアルカリ電池を検討していたが、Arduino の 5V からの電力供給と DC ジャックからの AC アダプター 5V1A による外部給電により、アルカリ電池が不要であることが実装時に確認されたため除外した。また、モータードライバー及び DC ジャックはピンヘッダーのはんだ付けを行う。加えて、ギアボックスは 0.01 μ F のコンデンサーと直接はんだ付けを行う。

次に、観覧車の設計について説明する。作成するために、3D プリンタを用いて作成する。実際に設計に使用するモデルデータを図 11 から図 16 に示している。観覧車の直径は約 16cm とし、ボタン電池付きレーザーを 45 度間隔で配置する。そして、観覧車を支える左右の支柱および土台を設計する。観覧車本体は、以下のパーツに分割して設計を行う。

- 土台 観覧車全体を支える部分であり、内部を空洞化させる。また、支柱を土台に差し込み、固定する役割も持つ。実際に設計する部品は図 11 の通りである。
- 支柱 観覧車の回転軸を左右から支える部品である。回転軸を安定して保持し、観覧車が滑らかに回転できるようにする。回転軸を上から挿入する形であり、軸が折れないようギアボックスモータに繋ぐ側の支柱の支える穴を大きくしている。実際に設計する部品は図 12 の通りである。

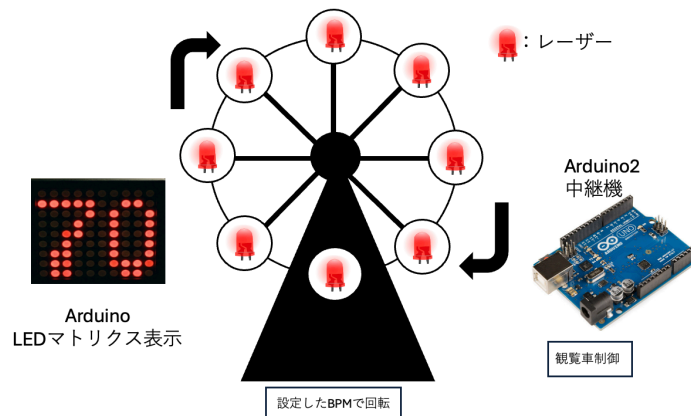


図 9 中継機デバイスの構成図

- 回転軸 ギヤボックスモータの回転を観覧車本体へ伝達する部品である。モータと接続し、観覧車全体を回転させる役割を持つ。支柱に固定できるよう、支柱の位置の軸部をへこませて固定できるようにしている。実際に設計する部品は図 13 の通りである。
- 回転リング 観覧車の円形部分であり、レーザー側とその反対側で 2 個作成する。モータによって回転し、レーザーによる光信号送信を行う部分でもある。角度 45 度おきにレーザーを設置する穴を用意し、そこにレーザーをはめ込み固定する。もう一つの円盤に電池基板を設置し電力供給する。実際に設計する部品は図 14, 15 の通りである。
- 受光部 レーザーの延長線上に設置する照度センサーの受光部である。回転リングから平行な位置に円盤を設置し、中央にギヤボックスモータを設置する台を設ける。90 度おきに円盤にくぼみを作り、センサーを設置できるようにした。受光部の円盤によりレーザー光が後ろに漏れないという設計となっている。実際に設計する部品は図 16 の通りである。

図～～～に実際に作成した中継機デバイスを示す。

2.2.3 内部設計

本節では中継機デバイスの内部設計について説明する。主に回転速度同期機構と LEDMatrix 表示機構のプログラムについて説明する。

回転速度同期機構の設計 まず、モータ制御の原理と BPM の変換について説明する。本デバイスのギヤボックスモータの回転速度を制御するために、Arduino の PWM(パルス幅変調) 機能を用いてモータの平均印加電圧を制御する。PWM とは、デジタル出力ピンから出力される短形波の HIGH 状態と LOW 状態の時間比率(デューティー比)を変化させることで、擬似的にアナログ電圧を生成する制御方式である。Arduino のデジタル出力ピンは 5V の HIGH か 0V の LOW のいずれかしか出力できないが、HIGH と LOW を高速に切り替えてデューティー比を調整することで、負荷に対して見かけ上の連続的な電圧を印加することが可能となる。

表 2 中継機デバイスを構成する部品

機材	品番	個数 [個]
Arduino Uno R4 WiFi	-	1
ルーター	-	1
ブレッドボード	-	1
赤色ドットレーザーモジュール	FU650AD5-C6	8
ボタン電池基板取付用ホルダー (CR2477 用)	CH031-2477LF	4
ボタン電池	CR2477	4
スライドスイッチ	SS-12D00-G5	4
モータードライバーモジュール	AE-DRV8835-S	1
ハイパワーギヤーボックス HE	72003	1
ジャンパーワイヤー	-	適宜
ブレッドボード用 2.1mm 標準 DC ジャック DIP 化キット	AE-DC-POWER-JACK- DIP	1
超小型スイッチング AC アダプター 5 V 1 A	-	1
QI ケーブル 2P-1Px2	311-262	1
抵抗器 10 Ω	-	1
コンデンサー 0.01 μF	-	1
コンデンサー 0.047 μF	-	1
有極性コンデンサー 100 μF	-	1

本デバイスは、Arduino の D5 および D6 ピンをモータードライバーモジュールに接続し、PWM 制御を行う構成としている。Arduino の analogWrite 関数は、引数として 0 から 255 の整数値を受け取り、この値に応じたデューティ比の矩形波を出力する。このとき引数が 0 のときデューティ比は常時 LOW であり、255 のときデューティ比は常時 HIGH となる。中間値では、引数に比例したデューティ比の矩形波が生成される。この関係を式で表すと、供給電圧を V_{cc} (定数 VCC)、プログラム上の出力値を $pwmValue$ とすると、モータに加わる平均電圧 $V_{average}$ は (1) 式で決定される。

$$V_{average} = V_{cc} \times \frac{pwmValue}{255} \quad (1)$$

2.3 楽器デバイス

果たしてデバイスなのか_____

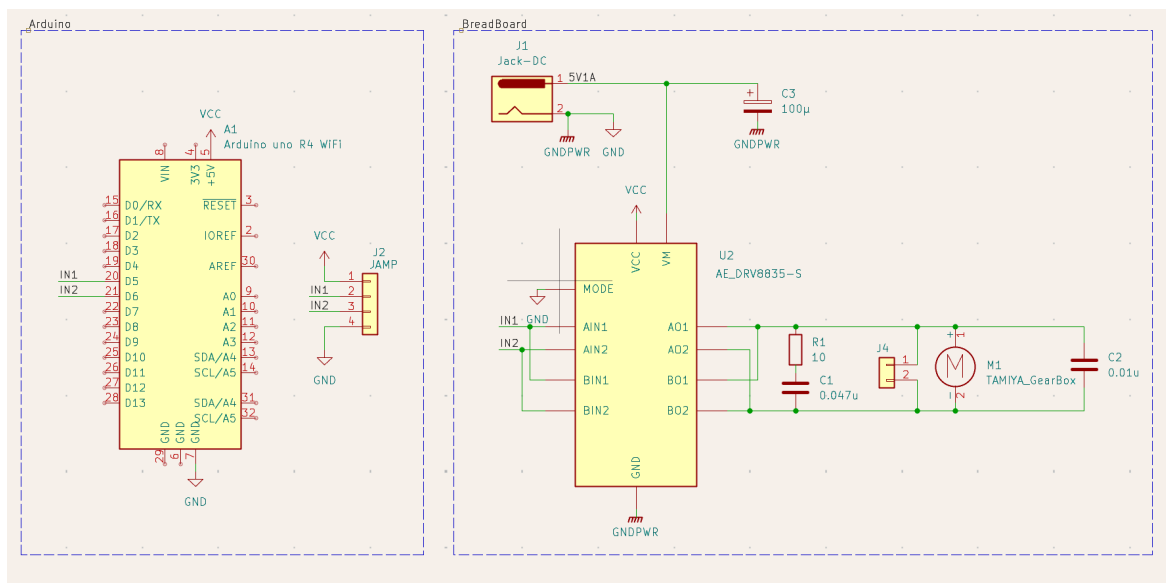


図 10 観覧車型中継機デバイス回路図

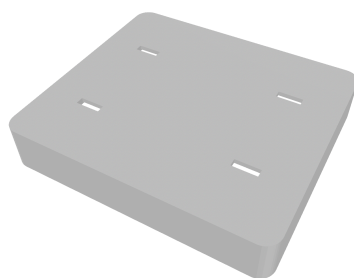


図 11 設計に使用するモデルデータ (土台部)

3 システム実装

本節では、担当したハードウェアの詳細設計と LEDMatrix での BPM 表示を行うプログラムについて説明する。

3.1 部品一覧

次に、楽器デバイスが受光するための部品について表 3 に示す。光センサモジュールとして CdS セルを使用する。CdS セルにレーザー光が照射された際の出力を Arduino のアナログピンへ接続することで、観覧車デバイスのレーザー光の通過を検知する。また、今回はドラム、ピアノ、ストリングス、フルートの 4 種類の楽器の音色を再現するためそれぞれに Arduino を使用し、光センサおよびブレッドボードも 4 台用意する。

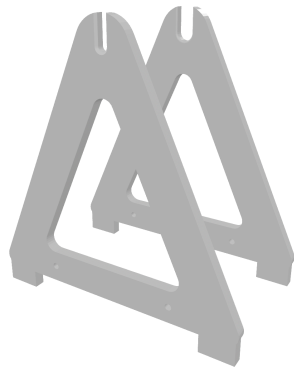


図 12 設計に使用するモデルデータ (支柱部)

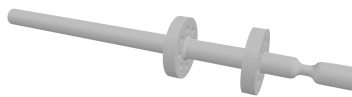


図 13 設計に使用するモデルデータ (回転軸部)

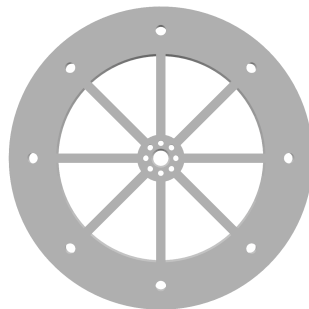


図 14 設計に使用するモデルデータ (回転リング部 1)

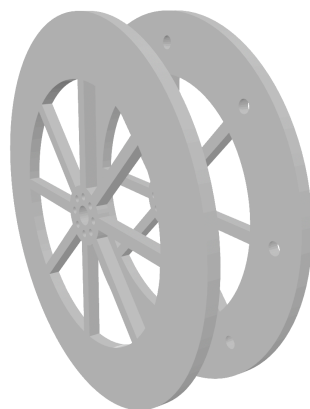


図 15 設計に使用するモデルデータ (回転リング部 2)

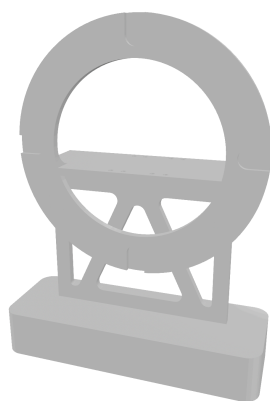


図 16 設計に使用するモデルデータ (受光部)

表 3 楽器デバイスに必要な機器

機材	品番	個数 [個]
Arduino UNO R4	-	4
CdS セル	GL5516	4
ブレッドボード	-	4
ジャンパーワイヤー	-	適宜

3.2 指揮デバイス

3.3 観覧車型中継機デバイス

3.4 レーザー光送信装置

レーザー光送信装置の詳細設計について説明する。図 17 について、回路図の通りにボタン電池とレーザーをはんだ付けする。これらを観覧車のリングに 45 度間隔に装着する。電源としては図 17 に示すようにボタン電池からの 3V 給電を行う。ここで、レーザーについて、データシートより定電流源回路を組み込んであるため、現在の設計であれば外部抵抗は必要とせず、素子を安全に利用することができる [?].

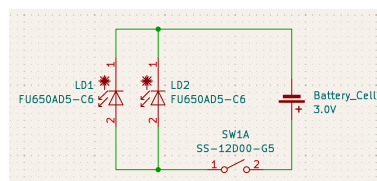


図 17 レーザー回路図

3.5 楽器デバイス受光部装置

楽器デバイスの受光部の回路図を図 18 に示す。電源仕様は、PC との USB 接続から電力を供給する。光センサは Arduino の 5V ピンから給電されるため、外部電源は不要である。また、光センサが光を受光した瞬間に A0 のアナログピンにセンサ値を出力する。このセンサ値の変化を Arduino で処理することで光を検知する仕様である。

次に、Display.cpp ファイルでは、表示文字の独自フォント定義、マトリクスへの描画処理、そして通信処理を行う。また、Display.h ファイルでは定数の定義と関数宣言、独自フォント定義などを行う。ここで、今回利用する Arduino のマトリクスは縦 12 横 8 の範囲で構成されている。この時、通常で定義されている文字列では文字の間隔が短くなることや、それに付随した可読性の低下が考えられる。よって、今回は 1 文字を縦 5 横 3 で再定義することにより文字同士の間隔を開け、可読性の担保を目指す。例として数字 0 を以下に示す。

```
{1,1,1,
 1,0,1,
 1,0,1,
 1,0,1,
 1,1,1}
```

また、LED マトリクスの描画処理は、受信した段階番号から BPM 値を参照し、整数を各桁に分割した後、独自定義した 3 × 5 のフォント配列を用いてフレームバッファに書き込むことで行う。

3.6.2 オブジェクト図 (クラス図)

LEDMatrix プログラムのクラス図は以下の図 19 の通りである。通信開始を行う Display.ino、通信受信処理と LED マトリクスへの描画処理を行う Display.cpp、および関数宣言やフォント定義を行う Display.h で構成される。各モジュール間の関係について、Display.ino から Display.cpp への関係は、表示処理を利用する依存関係である。また、Display.cpp から Display.h への関係は、関数宣言およびフォント定義を参照する依存関係である。

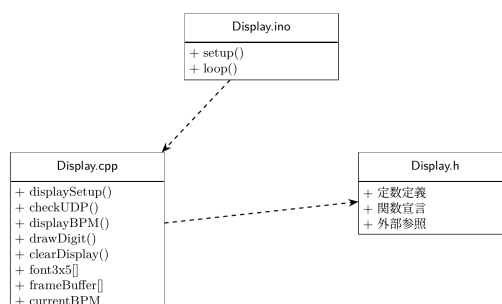


図 19 LEDMatrix のクラス図

3.6.3 関数設計

表 5 に今回用いる主な処理を行う関数を示している。以降では、各関数の具体的な動作を説明する。

はじめに、displaySetup 関数について説明する。この関数は、表 5 に示すように、LEDMatrix の初期化を行い、引数や戻り値はない。matrix.begin() を呼び出した後、消灯フレームを一度描画し

てから 200 ms 待機する。これは、begin() 直後の最初の loadFrame() は反映されない場合があるための対策である。

次に、checkUDP 関数について説明する。この関数は表 5 に示すように、中継機からのパケット確認と BPM 情報の値確認を行い、引数や戻り値はない。まず、中継機からのパケットを Udp.parsePacket() を用いて監視する。パケットが届いていることを確認できれば、uint8_t buffer[2] としてバイナリ形式で 2 バイトを読み取る。buffer[0] をヘッダ文字として判定し、ヘッダが'E' の場合は clearDisplay() を呼び出して LED を消灯する。また、ヘッダが'B' の場合は buffer[1] の段階番号を bpmTable[] で参照して BPM 値に変換し、値が更新されていれば描画更新処理を行う。

次に、displayBPM 関数について説明する。この関数は表 5 に示すように、BPM 数値のマトリクス描画を行い、引数としては、checkUDP 関数から保持した数値を用いる。まず、BPM 値を各桁ごとに分割し、drawDigit() を用いてフレームバッファに描画する。そして、uint32_t[3] 形式にビットパックした後、matrix.loadFrame() で描画を行う。

次に、drawDigit 関数について説明する。この関数は、表 5 に示すように、一桁の数字をフレームバッファに描画をし、引数としては描画する数字とフレームバッファ上の横方向開始位置を取る。まず、引数の数字が 0 から 9 の範囲内であるかを確認する。次に、縦方向の中央寄せオフセットを算出して配置する。その後、独自フォント配列から該当する数字のドットパターンを 1 ドットずつ読み取り、フレームバッファの指定位置に書き込む。

最後に、clearDisplay 関数について説明する。この関数は、表 5 に示すように、LEDMatrix を全消灯させる処理を行い、引数や戻り値はない。BPM 値を初期値にリセットし、全消灯フレームを matrix.loadFrame() で描画して表示のリセットを行う。

表 5 主要な処理を行う関数一覧

関数名	概要	引数	戻り値
displaySetup	LED マトリクスの初期化	なし	なし
checkUDP	UDP パケットの確認、BPM 更新・消灯処理	なし	なし
displayBPM	BPM 数値を LED マトリクスに描画	int bpm	なし
drawDigit	1 桁の数字をフレームバッファに描画	int digit, int x_offset	なし
clearDisplay	LED マトリクスを全消灯	なし	なし

3.7 Display.h

```

1 #ifndef DISPLAY_H
2 #define DISPLAY_H
3
4 #include <Arduino.h>
5 #include "Arduino_LED_Matrix.h" // Arduino UNO R4 WiFi 内蔵LEDマトリクス用ライブラリ
6 #include <WiFiS3.h> //UDP通信
7
8 // BPM定数定義
9 #define BPM_STEP_1 60

```

```

10 #define BPM_STEP_2 90
11 #define BPM_STEP_3 120
12 #define BPM_STEP_4 150
13 #define BPM_STEP_5 180
14
15 // 通信定数
16 #define LOCAL_PORT 2390 // UDP受信に利用するポート番号
17 #define HEADER_BPM 'B' // BPMデータ受信の識別ヘッダ
18 #define HEADER_END 'E' // 演奏終了の識別ヘッダ (LEDマトリクス消灯)
19
20 // 文字フォントの寸法
21 #define FONT_WIDTH 3 // 1文字の横ドット数
22 #define FONT_HEIGHT 5 // 1文字の縦ドット数
23
24 // LEDマトリクスの寸法 (Arduino UNO R4 WiFi 内蔵: 8×行12列)
25 #define MATRIX_ROWS 8
26 #define MATRIX_COLS 12
27
28 // 関数宣言
29
30 // LEDマトリクスの初期化 (matrix.begin())を本関数経由で呼ぶ。
31 void displaySetup();
32
33 // 中継機からのUDPパケット確認, BPM値の受信処理
34 void checkUDP();
35
36 // 受信したBPM値をLEDマトリクスに数値表示する (引数: int bpm)
37 void displayBPM(int bpm);
38
39 // LEDマトリクスを全消灯する (演奏終了時に使用)
40 void clearDisplay();
41
42 // 指定された1桁の数字を, 共有フレームバッファの x_offset 列から描画する
43 void drawDigit(int digit, int x_offset);
44
45 // 外部参照
46
47 // ×35 (横×3縦5) の独自数字フォント (0~9)
48 extern const uint8_t font3x5[10][15];
49
50 // LEDマトリクスインスタンス (Display.cpp で定義, .ino の setup() で begin() を呼ぶ)
51 extern ArduinoLEDMatrix matrix;
52
53 // UDP通信オブジェクト
54 extern WiFiUDP Udp;
55
56 #endif

```

3.8 Display.ino

```
1 #include "Display.h"
2
3 WiFiUDP Udp;
4
5 const char SSID[] = "hackathon001-WPA2";
6 const char PASS[] = "hackathon001";
7
8 // 静的IP設定 (指揮デバイスの送信先 ferrisWheelIP2 と一致)
9 IPAddress localIP(192, 168, 11, 10);
10 IPAddress gateway(192, 168, 11, 1);
11 IPAddress subnet(255, 255, 255, 0);
12
13 void setup() {
14     displaySetup();
15
16     if (SSID[0] != '\0') {
17         WiFi.config(localIP, gateway, subnet);
18         while (WiFi.begin(SSID, PASS) != WL_CONNECTED) {
19             delay(1000);
20         }
21     }
22     Udp.begin(LOCAL_PORT);
23 }
24
25 void loop() {
26     checkUDP();
27 }
```

3.9 Display.cpp

```
1 #include "Display.h"
2 #include "Arduino_LED_Matrix.h"
3
4 // 独自フォント定義
5 // font3x5[digit][row*FONT_WIDTH + col] : 1=点灯, 0=消灯
6
7 const uint8_t font3x5[10][15] = {
8     // 0
9     {1, 1, 1,
10      1, 0, 1,
11      1, 0, 1,
12      1, 0, 1,
13      1, 1, 1},
14     // 1
15     {0, 1, 0,
```



```

16 1, 1, 0,
17 0, 1, 0,
18 0, 1, 0,
19 1, 1, 1},
20 // 2
21 {1, 1, 1,
22 0, 0, 1,
23 1, 1, 1,
24 1, 0, 0,
25 1, 1, 1},
26 // 3
27 {1, 1, 1,
28 0, 0, 1,
29 0, 1, 1,
30 0, 0, 1,
31 1, 1, 1},
32 // 4
33 {1, 0, 1,
34 1, 0, 1,
35 1, 1, 1,
36 0, 0, 1,
37 0, 0, 1},
38 // 5
39 {1, 1, 1,
40 1, 0, 0,
41 1, 1, 1,
42 0, 0, 1,
43 1, 1, 1},
44 // 6
45 {1, 1, 1,
46 1, 0, 0,
47 1, 1, 1,
48 1, 0, 1,
49 1, 1, 1},
50 // 7
51 {1, 1, 1,
52 0, 0, 1,
53 0, 0, 1,
54 0, 0, 1,
55 0, 0, 1},
56 // 8
57 {1, 1, 1,
58 1, 0, 1,
59 1, 1, 1,
60 1, 0, 1,
61 1, 1, 1},
62 // 9
63 {1, 1, 1,

```

```

64     1, 0, 1,
65     1, 1, 1,
66     0, 0, 1,
67     1, 1, 1}
68 };
69
70 // bpmTable[stage-1] に対応するBPMを得る
71 static const int bpmTable[5] = {
72     BPM_STEP_1,    // 60
73     BPM_STEP_2,    // 90
74     BPM_STEP_3,    // 120
75     BPM_STEP_4,    // 150
76     BPM_STEP_5     // 180
77 };
78
79 // LEDマトリクス of インスタンス (8×行12列)
80 ArduinoLEDMatrix matrix;
81
82 // displaySetup (LEDマトリクスの初期化)
83 // Arduino_LED_Matrix の表示バッファ(framebuffer)はヘッダ内で static 定義
84 // されており, include する翻訳単位(.ino / .cpp)ごとに別実体となる. そのため
85 // 表示を駆動する割り込みを登録する begin() と, 描画を行う loadFrame() は
86 // 必ず同一ファイル(本 Display.cpp)から呼び出し, 同じ framebuffer を共有させる.
87 // .ino からは matrix を直接触らず, 本関数と displayBPM() を呼び
88
89 void displaySetup() {
90     matrix.begin();
91
92     // begin() 直後の最初の loadFrame は反映されない場合があるため,
93     // 消灯フレームを一度描画(ウォームアップ)してから安定するまで待機する.
94     uint32_t warmUp[3] = {0, 0, 0};
95     matrix.loadFrame(warmUp);
96     delay(200);
97 }
98
99 // 描画用共有フレームバッファ (MATRIX_ROWS行 × MATRIX_COLS列, 1=点灯 / 0=消灯)
100 // drawDigit() がここに書き込み, displayBPM() がまとめてレンダリングする
101 static uint8_t framebuffer[MATRIX_ROWS][MATRIX_COLS];
102
103 // 直近に表示中のBPM値 (更新検知用: 値が変化した時だけ再描画する)
104 static int currentBPM = -1;
105
106 // checkUDP
107 // buffer[0]はヘッダ('B'とか)
108 // buffer[1]は段階番号
109
110 void checkUDP() {
111     int packetSize = Udp.parsePacket();

```

```

112     if (packetSize <= 0) return;
113
114     uint8_t buffer[2] = {0};
115     int len = Udp.read(buffer, sizeof(buffer));
116     if (len < 2) return;
117
118     char header = (char)buffer[0];
119     uint8_t stage = buffer[1];
120
121     if (header == HEADER_END) {
122         clearDisplay();
123         return;
124     }
125
126     if (header != HEADER_BPM) return;
127     if (stage < 1 || stage > 5) return;
128
129     int bpm = bpmTable[stage - 1];
130
131     if (bpm != currentBPM) {
132         currentBPM = bpm;
133         displayBPM(bpm);
134     }
135 }
136
137 // displayBPM
138
139 void displayBPM(int bpm) {
140     // フレームバッファをすべて消灯にリセット
141     memset(frameBuffer, 0, sizeof(frameBuffer));
142
143     // BPM値を桁配列に分解
144     int digits[3]; // 最大3桁分
145     int numDigits;
146
147     if (bpm >= 100) {
148         numDigits = 3;
149         digits[0] = bpm / 100; // 百の位
150         digits[1] = (bpm / 10) % 10; // 十の位
151         digits[2] = bpm % 10; // 一の位
152     } else if (bpm >= 10) {
153         numDigits = 2;
154         digits[0] = bpm / 10; // 十の位
155         digits[1] = bpm % 10; // 一の位
156     } else {
157         numDigits = 1;
158         digits[0] = bpm; // 一の位のみ
159     }

```

```

160
161 // 横方向中央寄せの計算
162 // 各桁は幅FONT_WIDTH(3), 桁間スペース1px → 合計幅 = numDigits*3 + (numDigits-1)*1
163 int totalWidth = numDigits * FONT_WIDTH + (numDigits - 1) * 1;
164 int startX = (MATRIX_COLS - totalWidth) / 2; // 12列に対する左端オフセット
165
166 // 各桁をフレームバッファに描画 (桁ピッチ = 3px + 1px間隔 = 4px)
167 for (int i = 0; i < numDigits; i++) {
168     drawDigit(digits[i], startX + i * (FONT_WIDTH + 1));
169 }
170
171 // フレームバッファ×(812=96ドット)を uint32_t[3] 形式へパックする.
172 // ドット番号 n = row*12 + col (0~95) を, MSB側から順に詰める
173 // (Arduino_LED_Matrix の loadFrame が要求するビット配置).
174 uint32_t bitmap[3] = {0, 0, 0};
175 for (int row = 0; row < MATRIX_ROWS; row++) {
176     for (int col = 0; col < MATRIX_COLS; col++) {
177         if (frameBuffer[row][col]) {
178             int n = row * MATRIX_COLS + col;
179             bitmap[n / 32] |= (1UL << (31 - (n % 32)));
180         }
181     }
182 }
183
184 // LEDマトリクスへ描画
185 matrix.loadFrame(bitmap);
186 }
187
188 // drawDigit
189 // 引数 digit : 描画する数字 (0~9)
190 // x_offset : フレームバッファ上の描画開始列 (0~11)
191 // font3x5[digit] の 横×3縦5 ビットマップを, frameBuffer の
192 // (y_offset~, x_offset~) 領域に書き込む.
193 // 縦方向は (MATRIX_ROWS - FONT_HEIGHT)/2 行のオフセットで上下中央寄せ.
194
195 void drawDigit(int digit, int x_offset) {
196     // 範囲外の数字は無視
197     if (digit < 0 || digit > 9) return;
198
199     // 縦方向中央寄せオフセット: 8行に5行フォントを配置 → (8-5)/2 = 1行余白
200     const int y_offset = (MATRIX_ROWS - FONT_HEIGHT) / 2;
201
202     for (int y = 0; y < FONT_HEIGHT; y++) { // 縦5ドット
203         for (int x = 0; x < FONT_WIDTH; x++) { // 横3ドット
204             int col = x_offset + x;
205             int row = y_offset + y;
206             // バッファ境界チェック (端に近い場合の範囲外アクセス防止)
207             if (col >= 0 && col < MATRIX_COLS && row >= 0 && row < MATRIX_ROWS) {

```

```
208         frameBuffer[row][col] = font3x5[digit][y * FONT_WIDTH + x];
209     }
210 }
211 }
212 }
213
214 //追加 'E'を受信したら全消灯, currentBPMをリセット
215 void clearDisplay() {
216     currentBPM = -1;
217     uint32_t blank[3] = {0, 0, 0};
218     matrix.loadFrame(blank);
219 }
```

4 おわりに

これは $\text{T}_\text{E}\text{X}$ による報告書作成のテンプレートでした.

参考文献

- [1] 株式会社マーケットリサーチセンター, “音楽出版の日本市場(～2031年)、市場規模(パフォーマンス、同期、デジタル収益)・分析レポートを発表,” <https://www.atpress.ne.jp/news/4436457>, 2026/4/22 参照.
- [2] LP Information, “ライブコンサート市場占有率分析レポート,” <https://www.atpress.ne.jp/news/3304316>, 2026/4/22 参照.
- [3] 一般社団法人コンサートプロモーターズ協会, “ライブ市場調査 基礎推移表” <https://www.acpc.or.jp/marketing/transition/>, 2026/5/18 参照.
- [4] 谷口由貴, “ストーリーミングサービスが生み出した新たな音楽消費サイクル,” 博報堂 The Hakuhodo, <https://www.hakuhodo.co.jp/magazine/61505/>, 2019/9/12, 2026/7/7 参照.
- [5] Martin Kreuzer, Melanie Wald-Fuhrmann, Christian Weining, Martin Tröndle, and Hauke Egermann, “Western Classical Music Concerts Are More Immersive, Intellectually Stimulating, and Social, When Experienced Live Rather Than in a Digital Stream: An Ecologically Valid Concert Study on Different Modes of Liveness,” *Music & Science*, vol.8, 2025.
- [6] Natalia Cooper, Dimitrios Mileounis, Patrick Williams, and Frank P. Brooks, “The effects of substitute multisensory feedback on task performance and the sense of presence in a virtual reality environment,” *PLOS ONE*, vol.13, no.2, p.e0191846, 2018.
- [7] Martin, R., and Nielsen, N. Enacting Musical Aesthetics: The Embodied Experience of Live Music. *Music & Science*, 7, 2024
- [8] Yamaha Music Japan, “ENSPIRE PRO,” https://jp.yamaha.com/products/musical_instruments/pianos/disklavier/enspire_pro/features.html#product-tabs, 2026/5/19 参照.
- [9] 関根 伸也, “DMX512 信号システムなどの現状技術,” 電気設備学会誌, vol.41, no.72026, p.382-385, 2021
- [10] ReacTable Legacy, “Welcome|ReacTable Legacy,” <https://reactable.com/>, 2026/5/19 参照.

A 役割分担とスケジュール

このテンプレートでは、 $\text{T}_{\text{E}}\text{X}$ でガントチャートを描いてみました。

B プログラムソース

ソースの書き方のサンプルです。

```
1 // 光センサによる居眠り検知システム (Doze-detection system using optical sensors)
2 #define USE_ARDUINO_INTERRUPTS true
3 #include <PulseSensorPlayground.h>
```

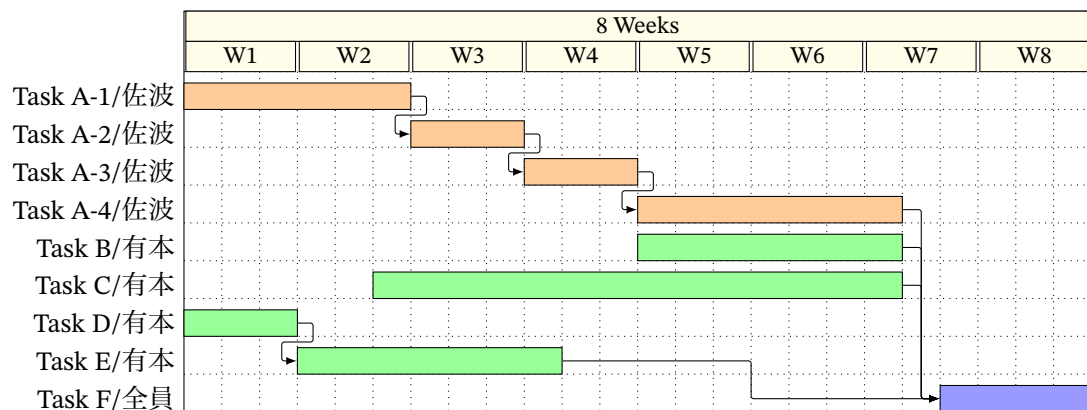


図 A.20 計画時のスケジュール

```

4  const int SENSOR_A = 0; //環境光(部屋の明るさ)
5  const int SENSOR_B = 1; //頭の動きを検知
6  const int SENSOR_Pulse = 2; //心拍を検知
7  const int IN1 = 10; //モータ
8  const int IN2 = 9; //モータ
9  const int check_range = 50; //センサーの値をチェックする範囲, つまり配列の大きさ 5秒間
10 const int A_dif = 2; //光センサAの値が一定になったと判別する際に用いる変数
11 const int Night_Mode_Value = 15; //夜モードになるための閾値
12 const float Judgment_Value = 4; //居眠りと判定するための変数 通常は環境光/3が頭の影
    (自宅環境)

```